INDUSTRIAL STRENGTH SOFTWARE SERVICES
TECHNICAL REPORT · ISSS-TR-0501

BATCHED MERGE FLUSHING

REMOVING PER-ENTRY FLUSH OVERHEAD FROM YABIIR'S COMPACTION PATH

AUTHOR
Javier Fernández

CONTACT
jfernand@me.com

ISSUED
2026-09-16

ABSTRACT

`yabiir` is a from-scratch Rust implementation of Bitcask, the log-structured hash-table key/value store described in Sheehy & Smith's 2010 paper. A `cargo flamegraph` capture of a synthetic mixed-load benchmark showed merge (compaction) taking over a second per pass on a modest dataset, with a large share of that time going to a flush syscall issued once per live entry copied — an artifact of a **correct** earlier fix (every write must flush, so a concurrent reader never sees stale bytes) applied more aggressively than merge actually needs. This report explains the problem, the batched-flush design that replaces it, why it doesn't weaken crash safety, and the controlled measurement used to verify it actually helps.

DOCUMENT CONTROL

REPOSITORY	`yabiir` — Rust, Bitcask log-structured storage engine
COMPONENT	`src/merge.rs`, `src/datafile.rs`
TRIGGER	`cargo flamegraph` capture of `src/bin/loadtest.rs` under concurrent load
VERIFICATION	Controlled A/B via `git stash`, real `loadtest` runs — see § 5

01 THE PROBLEM

Every write to a `yabiir` data file goes through `ActiveFile::append`, which writes the encoded entry into a buffered writer and — critically — flushes it before returning:

SRC/DATAFILE.RS — ACTIVEFILE::APPEND (BEFORE THIS CHANGE)

```rust
pub fn append(&mut self, encoded: &EncodedEntry) -> io::Result<(u32, u64, u64)> {
    let bytes = encoded.as_bytes();
    self.writer.write_all(bytes)?;
    self.writer.flush()?;
    let total_len = bytes.len() as u64;
    let value_pos = self.offset + (total_len - encoded.value_len() as u64);
    self.offset += total_len;
    Ok((self.file_id, value_pos, total_len))
}
```

> **WHY THE FLUSH IS THERE AT ALL**
> Reads (`get` / `fold`) go through a **separate** raw file handle (`DataFileSet::read_at`) that bypasses the writer's internal buffer. A `put` publishes a key by inserting into the in-memory keydir immediately after `append` returns — so if `append` didn't flush first, a concurrent `get` for that same key, arriving right after `put` returns, could read through the separate handle and see a short or stale file. The fix (landed in an earlier pass on this codebase) makes `flush` unconditional inside `append`, closing that race entirely.

That fix is correct for `put`. The problem is that `merge`'s output writer, `MergeOutputWriter::write_live_entry`, copies every still-live entry forward by calling that **same** `append` — meaning a merge pass paid one `flush` syscall per entry it copied, not because merge itself needed that guarantee per entry, but because it was reusing the code path that does.

A `cargo flamegraph` capture of `src/bin/loadtest.rs` (4 threads, 50,000 keys, mixed get/put/delete, merge running every 3 seconds) made the cost concrete:

SIGNAL	MEASUREMENT
`BufWriter::flush` under `put`	≈9.4% of `Engine::put`'s own sampled CPU time (`perf report`)
Merge duration per pass	1.0 – 1.5 s on a dataset accumulated over a few seconds of load
Foreground latency while merge ran	`get` / `put` / `delete` max latency spiked to 36 – 55 ms, vs. a 1 – 3 µs baseline

The third row is the one that actually motivated fixing this: the goal driving this whole investigation is **predictable** latency, not just throughput, and a background compaction pass stalling foreground operations by four orders of magnitude is exactly the kind of pause a synthetic load test exists to catch.

02 DESIGN

Merge doesn't need a flush after **every** entry. It writes an entry, then compare-and-swaps the keydir to point at it — nothing requires those two steps to be immediately adjacent, only that the keydir is never updated to point at bytes a reader could observe before they're actually flushed to the OS. That's a batch-level guarantee, not a per-entry one.

2.1 `ACTIVEFILE`: SEPARATING THE WRITE FROM THE FLUSH

`append` is split into an internal `write_unflushed` plus an explicit flush step. Two new methods expose the pieces separately, scoped `pub(crate)` since they're an internal contract between `merge.rs` and `datafile.rs`, not part of the crate's public surface:

SRC/DATAFILE.RS — THE SPLIT

```rust
fn write_unflushed(&mut self, encoded: &EncodedEntry) -> io::Result<(u32, u64, u64)> {
    let bytes = encoded.as_bytes();
    self.writer.write_all(bytes)?;
    let total_len = bytes.len() as u64;
    let value_pos = self.offset + (total_len - encoded.value_len() as u64);
    self.offset += total_len;
    Ok((self.file_id, value_pos, total_len))
}

pub fn append(&mut self, encoded: &EncodedEntry) -> io::Result<(u32, u64, u64)> {
    let result = self.write_unflushed(encoded)?;
    self.writer.flush()?;
    Ok(result)
}

pub(crate) fn append_buffered(&mut self, encoded: &EncodedEntry) -> io::Result<(u32, u64, u64)> {
    self.write_unflushed(encoded)
}

pub(crate) fn flush_only(&mut self) -> io::Result<()> {
    self.writer.flush()
}
```

`put` / `delete` still call `append` — unchanged, still flushes every time. Only `merge`'s writer switches to the unflushed variant.

2.2 `MERGEOUTPUTWRITER`: BATCHED FLUSH, REPORTED BACK TO THE CALLER

`write_live_entry` now writes unflushed, and flushes once every `MERGE_FLUSH_BATCH_SIZE` (256) entries — or immediately, for free, whenever a rotation boundary is hit, since rotation already calls `sync` (a full fsync, strictly stronger than a plain flush):

SRC/MERGE.RS — MERGEOUTPUTWRITER::WRITE_LIVE_ENTRY

```rust
fn write_live_entry(&mut self, entry: &Entry) -> io::Result<(u32, u64, bool)> {
    let encoded = format::encode_entry(&entry.key, &entry.value, false, entry.header.tstamp);
    let (file_id, value_pos, _) = self.current_data.append_buffered(&encoded)?;
    let hint = format::encode_hint(&entry.key, &entry.header, value_pos);
    self.current_hint.write_all(hint.as_bytes())?;
    self.pending_since_flush += 1;

    let flushed = if self.current_data.len() >= self.max_file_size {
        self.rotate()?; // fsyncs — strictly stronger than a flush
        true
    } else if self.pending_since_flush >= MERGE_FLUSH_BATCH_SIZE {
        self.flush_batch()?;
        true
    } else {
        false
    };

    Ok((file_id, value_pos, flushed))
}
```

2.3 DEFERRED REPOINTS

The caller (`merge_with_hook`) queues each entry's keydir repoint instead of applying it immediately, and only drains the queue when `write_live_entry` reports a flush actually happened — plus once more after `finish()`, for whatever's left in the final partial batch:

SRC/MERGE.RS — DEFERRED, BATCH-DRAINED REPOINTS

```rust
let mut pending: Vec<(Vec<u8>, KeydirEntry, KeydirEntry)> = Vec::new();

// ... inside the scan loop, per live entry ...
let (new_file_id, new_value_pos, flushed) = out.write_live_entry(&entry)?;
let old = KeydirEntry { file_id, value_pos: entry_value_pos, .. };
let new = KeydirEntry { file_id: new_file_id, value_pos: new_value_pos, .. };
pending.push((entry.key, old, new));

if flushed {
    apply_pending_repoints(keydir, &mut pending, &mut before_repoint);
}
// ... after the loop ...
out.finish()?; // flushes+fsyncs whatever's left in the final batch
apply_pending_repoints(keydir, &mut pending, &mut before_repoint);
```

Each `apply_pending_repoints` call still runs the race-window hook and the compare-and-swap exactly as before (`docs/bitcask-implementation-plan.md` §7.3) — a racing concurrent `put` still wins, and merge's now-stale copy is simply left orphaned for the next merge pass to clean up. Batching changes **when** a given entry's repoint happens, never **how** it's decided.

03 CRASH SAFETY

> THIS DOES NOT WEAKEN DURABILITY
> Old input files are removed only after `out.finish()` has fully flushed **and fsynced** every output file — that ordering is unchanged by this work. If the process crashes at any point during the write phase, whether flushing per-entry or per-batch, the untouched old input files are still on disk, and a subsequent recovery falls back to them as if the merge had never started. Batching only changes how much of merge's **own** in-progress work is discarded and redone by the next attempt — it never risks any data a `put` or `delete` call already returned `Ok` for.

This was checked deliberately, not assumed: see the reasoning trail in `src/merge.rs`'s module documentation, and the durability discussion that preceded implementing this change.

04 VERIFICATION

4.1 CORRECTNESS

- The full test suite (53 library tests plus `tests/model.rs`'s property-based model test) passes unchanged.
- `race_concurrent_put_during_merge_does_not_lose_the_write` — the deterministic test pinning down plan §7.3's concurrent-write-during-merge guarantee — run 20 times back to back with no failures. This test's paused key is, in its own configuration, always alone in its batch (a tiny `max_file_size` forces a rotation, hence a flush, after every single entry), so its timing is unaffected by batching in the general case.
- `tests/model.rs`'s proptest run at 2,000 cases (10× the default) with no failures.

4.2 PERFORMANCE — A CONTROLLED A/B, NOT A SINGLE NUMBER

The first attempt to verify this used `benches/merge.rs`'s existing Criterion suite and got a confusing, mixed result: some data points faster, some slower, by 5–25%. The cause: that suite deliberately sets `max_file_size: 1` to control the live/dead entry ratio and input file count precisely — but at `max_file_size: 1`, **every** entry write already exceeds the rotation threshold, so `MergeOutputWriter` rotates (and fsyncs) on every single entry regardless of this change. The 256-entry batch counter this report adds is never reached; that benchmark suite structurally cannot exercise this optimization.

> **A BENCHMARK THAT CAN'T SEE THE THING YOU CHANGED**
> This is worth stating plainly: `benches/merge.rs` remains a good measure of merge's cost as a function of dead-entry ratio and input file count, but it is **not** evidence either way for this specific change. Treating its noisy, contradictory numbers as a verdict here would have been a mistake — the honest move was recognizing the benchmark's own configuration made it blind to the change, not rationalizing the noise.

The real check used `src/bin/loadtest.rs` directly, at the realistic default `max_file_size` (64 MiB) the flamegraph capture in § 1 used, with a controlled comparison: `git stash` isolated this change from everything already committed, so both runs sat on the **identical** base (including the earlier active-file-read-lock removal and keydir hasher swap — separate fixes from the same profiling effort), differing only in whether merge's flushes were batched.

MERGE PASS	WITHOUT BATCHING	WITH BATCHING
1st	2.25 s	2.16 s
2nd	3.94 s	3.32 s
3rd	5.44 s	3.86 s
4th	— (window ended)	3.99 s

Same 20-second load window, same 3-second merge interval, both runs: three merge passes completed without batching, four completed with it — and every comparable pass was faster with batching than without. Pass duration still climbs over the window in both cases (more data accumulates between merges as the run progresses, and neither run's merge is finished before the next 3-second timer fires), but batching is measurably, consistently ahead throughout.

05 WHAT THIS DOESN'T ADDRESS

- **Merge's growing-duration trend itself.** Each successive pass in § 5.2 takes longer than the last, in both columns — a `merge_interval_secs: 3` cadence that can't keep up with the accumulation rate at these throughput levels is a scheduling/tuning question, not something this change targets.
- **`put`'s own per-call flush cost.** The ≈9.4% flush overhead cited in § 1 is `put`'s, not merge's, and `put` still flushes on every call — for correctness reasons unrelated to merge. A general fix (batching several concurrent callers' flushes together, "group commit") was discussed separately and deliberately deferred: it needs each caller to still block until **its own** data's shared flush completes before returning, which is a real design exercise, not a mechanical change like this one.
- **The known merge/recovery file-ordering gap** documented in `src/merge.rs`'s own module comment (a key written **truly concurrently** with a merge touching it can still be misordered by a later recovery) is unrelated to this change and remains exactly as it was.

06 ADDENDUM, 2026-11-12: WHAT CHANGED SINCE

This report describes the change as it was made, and the names it uses are the names of that day. The design it describes is unchanged: merge writes its output unflushed, flushes once per 256 entries or when an output file fills up, and repoints a key only after the flush that covers its copy.

The names moved. `MergeOutputWriter` is `OutputWriter` in `crates/yabiir/src/merge/output.rs`; the deferred repoints are `PendingRepoints` in `merge/repoints.rs`; `ActiveFile::flush_only` is `ActiveFile::flush`. Hint files have a writer of their own, `merge/hint.rs`.

Merge reads faster too. The input side had the mirror image of the problem this report fixed on the output side: each entry was read with two `read` syscalls on an unbuffered file, and a whole input file was held in memory before any of it was copied. Input is now read through one 256 KiB buffer, an entry at a time. On the load test of § 5 (4 threads, a merge every 3 seconds) a pass that took 2.1–2.6 s takes 0.6–1.1 s.

The ordering gap is closed. The last section calls the merge/recovery file-ordering gap unrelated and unchanged. It is fixed: a merge now closes the active file and reserves the ids of its output files, in one step under the writer's lock, below the id of every file written afterwards. A write made during a merge is replayed after the merge's copy of the same key, and stays the last write after a reopen. The module documentation of `src/merge/mod.rs` has the argument.